INDIA.RUNS · THE DATA & AI CHALLENGE

Intelligent Candidate Discovery & Ranking

Ranking candidates the way a great recruiter would — not by matching keywords, but by understanding who actually fits the role.

TEAM NAME	[FILL IN — your team name]
TEAM LEADER NAME	Rakshitha
PROBLEM STATEMENT	Build an AI system that ranks 100,000 candidates against a job description by genuine fit — not keyword overlap — within a 5-min / 16GB / CPU-only budget.

SOLUTION OVERVIEW

What we built, and why it's different

A fully explainable, rule-based ranking pipeline that reads a candidate's entire profile — not just their skills list or job title — and scores them against the JD's stated requirements, its explicit disqualifiers, and live platform behavior signals.

- Reads career-history text, not just declared skills — catches Tier-5 candidates who never wrote "RAG" but clearly built one.
- Applies the JD's own stated disqualifiers as explicit rules, instead of a generic skill-match score.
- Multiplies in behavioral availability (recency, response rate) — a perfect-on-paper but inactive candidate is down-weighted.
- 100% rule-based and auditable: every score traces back to a sentence in the profile, with zero hallucination risk.

AT A GLANCE

100,000

candidates scored

~23 sec

full-pool runtime, CPU only

0

honeypots in final top 100

7

scoring components, fully explainable

JD UNDERSTANDING & EVALUATION

Turning a long, opinionated JD into structured requirements

Must-have	Production embeddings/retrieval, vector DB or hybrid search, strong Python, ranking-evaluation experience (NDCG / MRR / MAP / A-B testing).
Sweet spot, not a wall	5-9 yrs experience, peak credit at 6-8 — JD explicitly says this is a range, not a hard cutoff.
Explicit disqualifiers	Research-only background, LangChain-only "AI experience" under 12 months, architects with no recent code, consulting-only careers, CV/speech/robotics without NLP.
Geography	Pune/Noida preferred; Hyderabad, Mumbai, Delhi NCR explicitly welcomed; relocation-willing candidates considered case-by-case.

Beyond the skills list: every requirement above is searched across the full profile text — headline, summary, and every career-history description — so candidates who demonstrate fit through their actual work (not their keyword list) are still found.

RANKING METHODOLOGY

How candidates are retrieved, scored, and combined

Score (75%)

- 45% hard-skill match (4 must-have groups)
- 15% nice-to-have match (fine-tuning, LTR, HR-tech, scale, OSS)
- 15% experience-band fit (peak 6-8 yrs)

Modifiers

- 15% location/relocation fit
- 10% production-shipping language signal
- x Disqualifier penalties (multiplicative, JD-specific)

Final pass

- x Behavioral-availability multiplier (redrob_signals)
- x Honeypot / data-quality penalty
- Rescale top 100 to a differentiated 0.40-0.99 band

No models, no GPUs, no hosted LLM calls during ranking — every component is a deterministic, auditable function of the profile and JD text.

EXPLAINABILITY & DATA VALIDATION

No black box, no hallucinated reasoning

- Every reasoning string is built directly from the same matched keyword groups and signals that produced the score — there is no separate "explain after the fact" LLM step that could invent a justification.
- Reasoning never mentions a skill or fact that isn't present in the candidate's own profile text.
- Inconsistent or low-quality profiles (e.g. "expert" proficiency claimed with 0 months of use, or stated experience that doesn't match the sum of career-history durations) are flagged as honeypot/data-quality risks and pushed to the bottom rather than silently trusted.

ON THE FULL 100K POOL

68

candidates flagged with internally inconsistent profiles — close to the ~80 documented honeypots

0

of those flagged candidates appear in the final top 100 — well under the 10% disqualification threshold

END-TO-END WORKFLOW

From JD input to ranked candidate output

1

Parse JD

JD requirements encoded as keyword groups, experience band, geography rules, and explicit disqualifiers.

>

2

Stream candidates

candidates.jsonl read line-by-line; each profile's full text and redrob_signals scored independently.

>

3

Score & penalize

Weighted base score, multiplicative disqualifier penalties, behavioral multiplier, honeypot check.

>

4

Keep top 100

A 100-item min-heap tracks the running top 100 — O(1) memory per candidate, single pass.

>

5

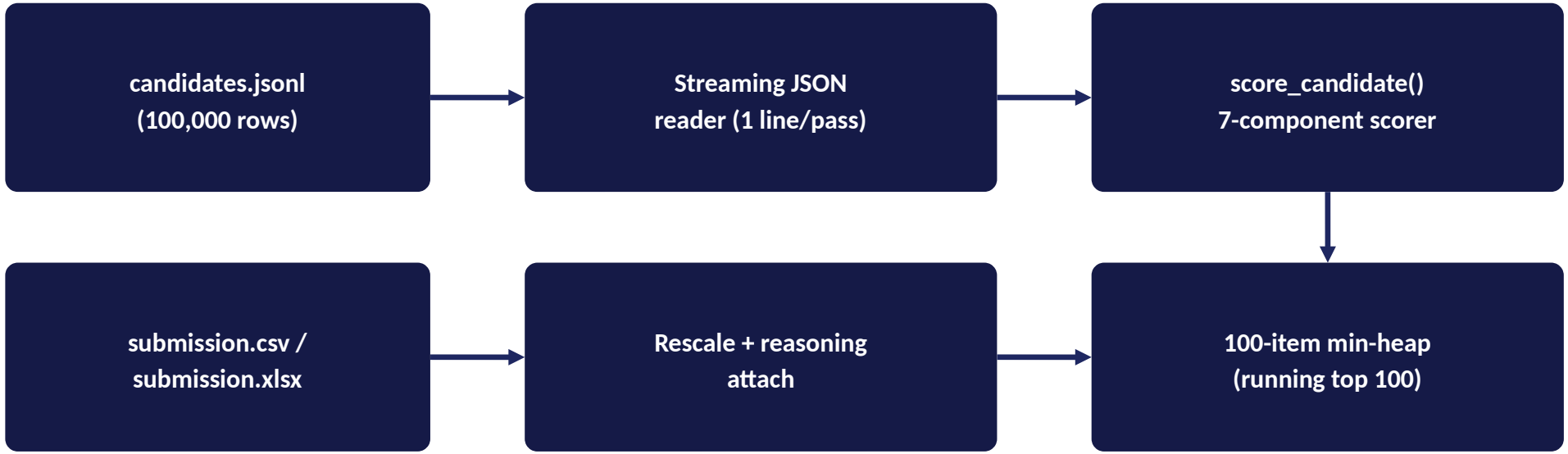
Rescale & explain

Scores rescaled to a differentiated 0.40-0.99 band; reasoning string attached per candidate.

```
Single command, single pass, no manual edits: python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

SYSTEM ARCHITECTURE

Single-pass, streaming, stateless scorer



No GPU, no network calls, no external services in the hot path. Memory footprint stays flat regardless of pool size — only the current line and the 100-item heap are held at once.

RESULTS & PERFORMANCE

Meeting the constraints, with room to spare

~23 sec

to score all 100,000 candidates, CPU only

< 1%

of the 5-minute compute budget used

0

honeypot candidates in the final top 100

100%

of rows pass validate_submission.py cleanly

WHY THIS MEETS THE COMPUTE CONSTRAINT

- No model inference at all — scoring is keyword/feature lookup plus arithmetic, so there's no GPU dependency to begin with.
- Single streaming pass with O(1) memory per row means the approach scales linearly and predictably well past 100K candidates.
- Deterministic and reproducible: the exact same command and code run inside Stage 3's sandboxed Docker container will produce the identical submission.csv.

TECHNOLOGIES USED

Deliberately minimal — and why

Python 3 (stdlib only)	json, csv, heapq, re, argparse, gzip, datetime — the entire ranking pipeline has zero external dependencies, eliminating environment-mismatch risk at Stage 3 reproduction.
openpyxl	Used only in the optional csv_to_xlsx.py step, to satisfy the submission portal's .xlsx upload requirement.
Google Colab	Hosts the sandbox notebook — a zero-setup, free way for organizers to verify the ranker runs end-to-end on a small sample.
Claude (Anthropic)	Used for architecture discussion, scoring-logic design, and code review during development. No candidate data was ever sent to a hosted LLM as part of the ranking step itself.

SUBMISSION ASSETS

Everything you need to verify this submission

GitHub repository

[FILL IN — https://github.com/your-username/your-repo]
Clean, complete, working code — README has the single reproduce command.

Sandbox (Colab)

[FILL IN — your Colab share link]
Runs rank.py end-to-end on a 50-candidate sample, CPU only, in seconds.

Ranked output file

submission.xlsx (and submission.csv) — top 100 candidates, validated against validate_submission.py with zero errors.

Thank you

A ranker that reads a resume the way a recruiter would — and can show its work.